SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems.(BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:25**

Duration : 1 Hour

Analytical Problem

Code of Conduct:

I **J. SAI DHEERAJ (192311191)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **J. Sai Dheeraj**

SIMATS ENGINEERING

ASSESSMENT TOOL

[1.] Consider the NFA given below which accepts some language L over the alphabet Σ={a,b}. Construct a DFA that represents the same language L

Concept

Every NFA has an equivalent DFA. The subset construction method is used: each DFA state is a set of NFA states, the start state is {q0}, and a DFA state is final if it contains at least one NFA final state.

Given NFA (read from the figure)

States = {q0, q1, q2}, Start state = q0, Final state = q2.

State	a	b
→ q0	{q0, q1}	{q2}
q1	{q0}	{q1}
* q2	∅	{q0, q1}

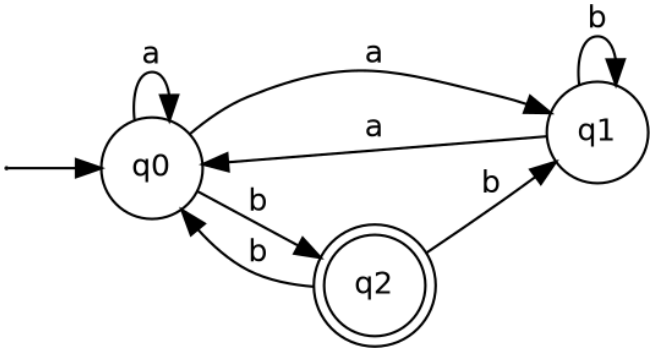


Fig 1.1 Given NFA

Subset Construction

- **A = {q0}:** δ(A,a) = {q0,q1} = B ; δ(A,b) = {q2} = C
- **B = {q0,q1}:** δ(B,a) = {q0,q1}∪{q0} = {q0,q1} = B ; δ(B,b) = {q2}∪{q1} = {q1,q2} = D
- **C = {q2}:** δ(C,a) = ∅= E (dead state) ; δ(C,b) = {q0,q1} = B
- **D = {q1,q2}:** δ(D,a) = {q0}∪∅= {q0} = A ; δ(D,b) = {q1}∪{q0,q1} = {q0,q1} = B
- **E = ∅:** δ(E,a) = δ(E,b) = ∅= E

DFA Transition Table

DFA State	a	b	NFA set
→ A	B	C	{q0}
B	B	D	{q0,q1}
* C	E	B	{q2}
* D	A	B	{q1,q2}
E (dead)	E	E	∅

Final states of the DFA: C and D (they contain q2).

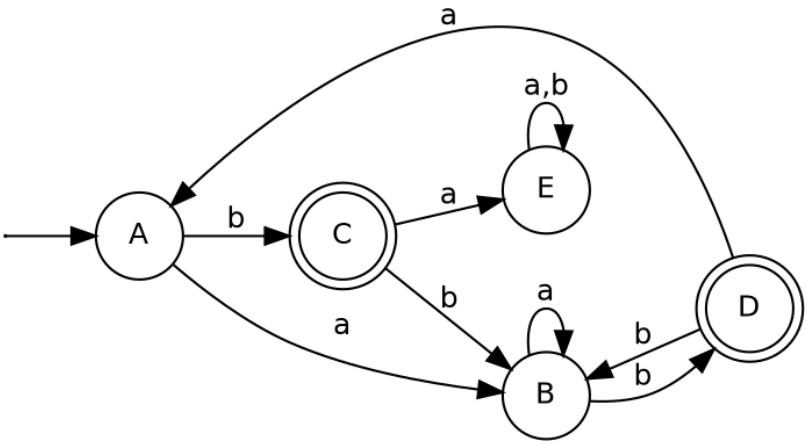


Fig 1.2 Equivalent DFA

Result: The DFA M = ({A,B,C,D,E}, {a,b}, δ, A, {C,D}) accepts exactly the language L of the given NFA.

[2.] Let L be the language of words of length atleast 2 whose last two letters are difference from each other. For example abaab is in L but not bb, Construct an NFA for L, Then draw the equivalent DFA

Concept

Over Σ={a,b}, the last two letters differ exactly when the string ends in ab or ba. The NFA loops on any symbol in q0 and then guesses (non-deterministically) that the last two symbols have begun. Because the pattern needs two symbols, every accepted string automatically has length at least 2.

Step 1: NFA for L

N = (Q, Σ, δ, q0, F) with Q = {q0, q1, q2, q3}, Σ = {a, b}, start = q0, F = {q3}.

State	a	b
→ q0	{q0, q1}	{q0, q2}
q1	∅	{q3}
q2	{q3}	∅
* q3	∅	∅

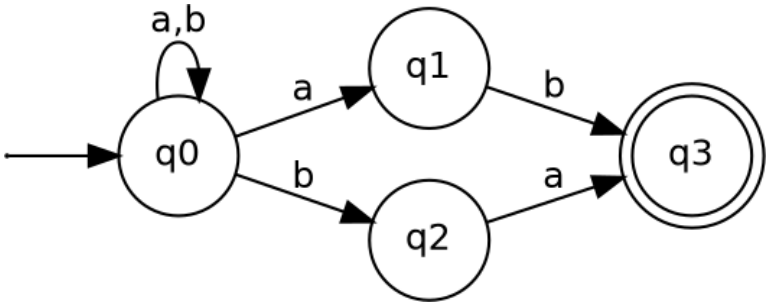


Fig 2.1 NFA for L

Check with abaab: q0 –a→ q0 –b→ q0 –a→ q0 –a→ q1 –b→ q3. The path ends in the final state q3, so abaab ∈ L. For bb: reaching q3 needs the last two symbols to be ab or ba, so no path ends in q3 and bb ∉ L.

Step 2: Subset Construction

DFA State	NFA set	on a	on b
→ A	{q0}	B	C
B	{q0,q1}	B	D
C	{q0,q2}	E	C
* D	{q0,q2,q3}	E	C
* E	{q0,q1,q3}	B	D

Where A={q0}, B={q0,q1}, C={q0,q2}, D={q0,q2,q3}, E={q0,q1,q3}. Final states are D and E (they contain q3).

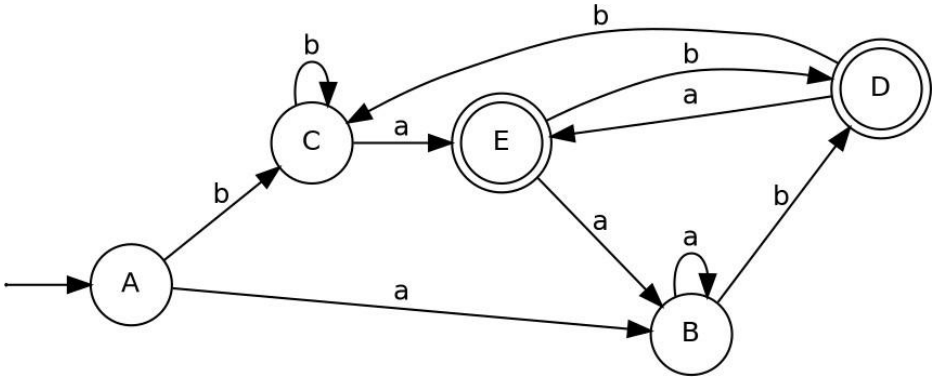


Fig 2.2 Equivalent DFA for L

Result: The DFA has 5 states with final states {D, E} and accepts every string over {a,b} that ends in ab or ba.

[3.] Design a Nondeterministic Finite Automata (NFA) that takes a binary string as input and accepts the string only if the last two digits are the same (either 00 or 11). Write the formal definition of the NFA. Also show that the string 01011 is accepted by the NFA

Concept

The language is L = { w ∈ {0,1}* | w ends with 00 or 11 }. The NFA stays in q0 reading any symbol, then guesses that the final two symbols are about to be read, going along the 00 branch or the 11 branch.

Formal Definition

N = (Q, Σ, δ, q0, F), where

- Q = {q0, q1, q2, q3, q4}
- Σ = {0, 1}
- q0 is the start state
- F = {q2, q4}
- δ is given by the transition table below

State	0	1
→ q0	{q0, q1}	{q0, q3}
q1	{q2}	∅
* q2	∅	∅
q3	∅	{q4}
* q4	∅	∅

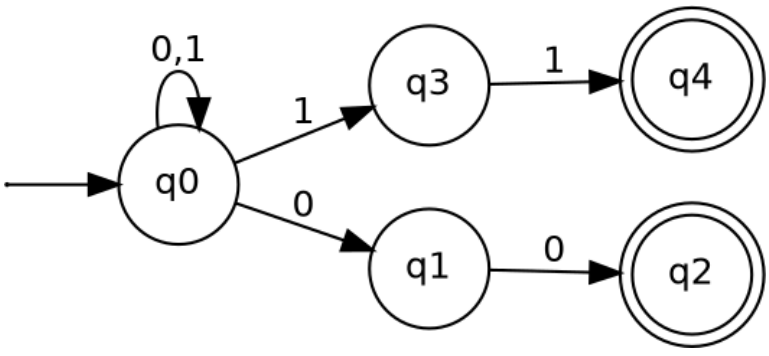


Fig 3.1 NFA accepting strings ending in 00 or 11

Acceptance of the string 01011

Input 0 1 0 1 1. One accepting computation path is:

Step	Current state	Symbol read	Next state
1	q0	0	q0 (stay in loop)
2	q0	1	q0 (stay in loop)
3	q0	0	q0 (stay in loop)
4	q0	1	q3 (guess 11 begins)
5	q3	1	q4 (final)

In δ-notation: $\delta(q0,0)=q0 \rightarrow \delta(q0,1)=q0 \rightarrow \delta(q0,0)=q0 \rightarrow \delta(q0,1)=q3 \rightarrow \delta(q3,1)=q4$.

Conclusion: All five symbols are consumed and the NFA reaches $q4 \in F$, so the string 01011 is accepted (its last two digits are 11).

[4.] Consider the NFA given below which accepts the language epresenting the set of all strings having 001 as a substring over the alphabet Σ={0,1}. Construct a DFA that represents the same language.

Concept

The standard NFA for "contains 001 as a substring" loops in q0 on 0,1, moves to q1 on 0, to q2 on 0, to q3 on 1, and then loops in the final state q3 on 0,1. The figure was not included with the question, so this standard NFA is used. The DFA is obtained by subset construction

Given NFA

State	0	1
→ q0	{q0, q1}	{q0}
q1	{q2}	∅
q2	∅	{q3}
* q3	{q3}	{q3}

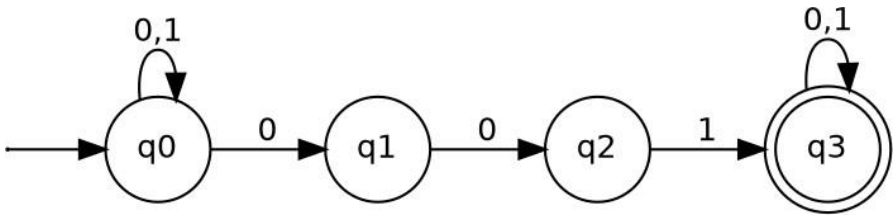


Fig 4.1 NFA for strings containing 001

Subset Construction

DFA State	NFA set	on 0	on 1
→ A	{q0}	B	A
B	{q0,q1}	C	A
C	{q0,q1,q2}	C	D
* D	{q0,q3}	E	D
* E	{q0,q1,q3}	F	D
* F	{q0,q1,q2,q3}	F	D

Final states: D, E and F (each contains q3).

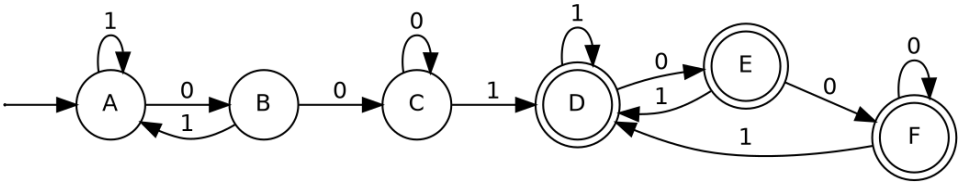


Fig 4.2 Equivalent DFA (6 states)

Minimised DFA

States D, E and F are all final and every one of them stays among the final states on both 0 and 1, so they are equivalent and can be merged into a single state S3. The minimal DFA has 4 states.

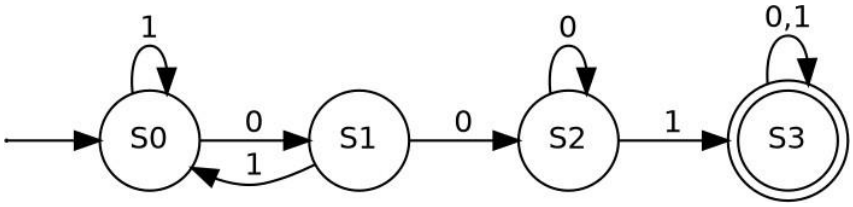


Fig 4.3 Minimal DFA (S0=A, S1=B, S2=C, S3={D,E,F})

Result: The DFA accepts exactly the strings over {0,1} that contain 001 as a substring.

[5.] A NFA with ε-transitions is given below. Find ε-closure for each state and Design an equivalent DFA which accepts the same language. Draw its transition table and mark the initial and final states

Given ε-NFA (read from the figure)

States = {q0, q1, q2}, Σ = {0, 1}, Start = q0, Final = q2. Transitions: q0 loops on 1, q1 loops on 0, q2 loops on 0,1; ε-moves q0→q1 and q1→q2.

State	0	1	ε
→ q0	∅	{q0}	{q1}
q1	{q1}	∅	{q2}
* q2	{q2}	{q2}	∅

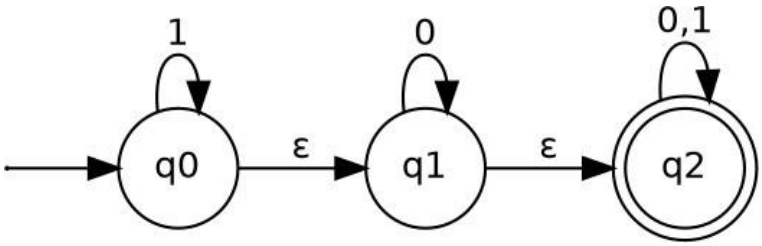


Fig 5.1 Given NFA with ε-transitions

Step 1: ε-closures

State	ε-closure
q0	ε-closure(q0) = {q0, q1, q2}
q1	ε-closure(q1) = {q1, q2}
q2	ε-closure(q2) = {q2}

Step 2: Subset Construction

Start state of the DFA = ε-closure(q0) = {q0, q1, q2} = A.

- **A = {q0,q1,q2}:** on 0: ε-closure(δ(q0,0) ∪δ(q1,0) ∪δ(q2,0)) = ε-closure({q1,q2}) = {q1,q2} = B
- on 1: ε-closure(δ(q0,1) ∪δ(q2,1)) = ε-closure({q0,q2}) = {q0,q1,q2} = A
- **B = {q1,q2}:** on 0: ε-closure({q1,q2}) = {q1,q2} = B ; on 1: ε-closure({q2}) = {q2} = C
- **C = {q2}:** on 0: {q2} = C ; on 1: {q2} = C

Step 3: DFA Transition Table

DFA State	0	1	Remark
→ * A = {q0,q1,q2}	B	A	Initial and Final

* B = {q1,q2}	B	C	Final
* C = {q2}	C	C	Final

Initial state: A. Final states: A, B, C (each contains q2).

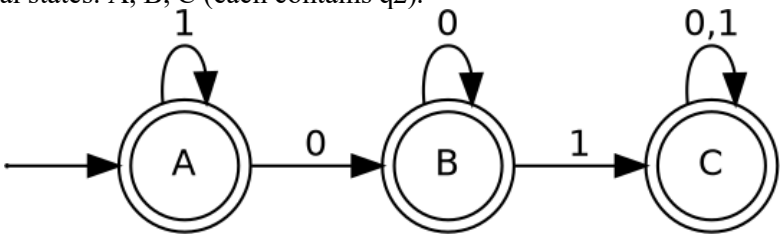


Fig 5.2 Equivalent DFA

Result: The language is 1*0*(0+1)*. Every state of the DFA is final, so it accepts all strings over {0,1} (including ε), i.e. L = Σ*. It can therefore be minimised to a single accepting state.